

Spiegazione dei Principi di Test

Prima di procedere con i file, è importante comprendere i principi di test richiesti:

A-TRIP:

- **Automatic:** I test devono essere automatizzati e non richiedere intervento manuale
- **Thorough:** I test devono coprire tutte le funzionalità in modo completo
- **Repeatable:** I test devono produrre gli stessi risultati in qualsiasi ambiente
- **Independent:** I test non devono dipendere l'uno dall'altro
- **Professional:** I test devono essere sviluppati con lo stesso livello di qualità del codice di produzione

RIGHT-BICEP:

- **Right:** Verifica che i risultati siano corretti
- **Boundary:** Verifica i casi limite (es. 0, 1, 1000, 1001)
- **Inverse:** Verifica le relazioni inverse (es. convertire e poi riconvertire)
- **Cross-check:** Verifica i risultati usando metodi alternativi
- **Error:** Verifica che il codice gestisca correttamente gli errori
- **Performance:** Verifica le prestazioni del codice

Ora vediamo ogni file richiesto.

Questo è il file di intestazione richiesto che verrà utilizzato come header in tutti i file sorgente e di test, secondo le specifiche fornite nel file checkstyle.xml.

///// FILE /////

Questo è il file di configurazione Maven (pom.xml). Include:

- Configurazione del progetto e dipendenze
- Plugin per l'analisi statica (checkstyle)
- Plugin per il code coverage (JaCoCo)
- Configura la soglia minima di copertura del codice all'85%
- Supporto per Coveralls per la pubblicazione dei report di copertura
- Plugin Badger per generare badge per il README

///// FILE /////

Questo è il file di configurazione per l'analisi statica del codice con Checkstyle. Include controlli per:

- Verificare che ogni file abbia l'intestazione corretta

- Limitare la complessità delle espressioni booleane
- Limitare la complessità ciclomatica (max 12)
- Controllare la lunghezza dei metodi
- Verificare che i blocchi catch non siano vuoti
- Evitare gli import con '*'
- Vietare import illegali
- Assicurare che tutte le istruzioni condizionali abbiano parentesi graffe
- Controllare la lunghezza dei file
- Controllare la lunghezza delle righe
- Vietare l'uso di caratteri di tabulazione

//// FILE ///

Questo è il file .gitignore che specifica quali file e directory devono essere ignorati dal sistema di controllo versione Git. Include:

- La directory 'target/' dove vengono compilati i file
- File di configurazione IDE come IntelliJ IDEA, Eclipse e VS Code
- File temporanei di sistema come .DS_Store per Mac

//// FILE ///

Questo è il file README.md che fornisce informazioni sul progetto. Include:

- Badge per lo stato della build, copertura del codice e risultati di checkstyle
- Descrizione del progetto
- Funzionalità principali
- Esempi di utilizzo
- Informazioni sugli sviluppatori
- Sostituisci "YourUsername" con il tuo nome utente GitHub reale quando utilizzi questo file

//// FILE ///

Questo è il file di configurazione per GitHub Actions che imposta la Continuous Integration (CI). Il workflow:

- Si attiva su push e pull request nei branch main/master
- Esegue su un ambiente Ubuntu aggiornato
- Configura JDK 8
- Esegue il build con Maven
- Esegue i test e genera report di copertura con JaCoCo
- Pubblica i report di copertura su Coveralls (richiede la configurazione del token nelle impostazioni del repository GitHub)

//// FILE ////

Questa è la classe principale App.java che contiene il metodo main per eseguire l'applicazione da riga di comando. La classe:

- Verifica che vengano passati argomenti
- Prova a convertire l'argomento in un numero intero
- Utilizza RomanPrinter per convertire e stampare il numero romano
- Gestisce eventuali eccezioni di formato numero e argomenti invalidi

//// FILE ////

Questa è la classe IntegerToRoman.java che implementa la conversione da numeri arabi a numeri romani. La classe:

- Verifica che il numero sia nell'intervallo valido (1-1000)
- Utilizza il metodo di sottrazione con array di valori e simboli
- Implementa le regole di notazione romana (es. IV per 4, IX per 9, ecc.)
- Restituisce la rappresentazione in numeri romani come stringa

//// FILE ////

Questa è la classe RomanPrinter.java che implementa la stampa dei numeri romani in ASCII art. La classe:

- Verifica che il numero sia nell'intervallo valido (1-1000)
- Converte il numero in romano utilizzando IntegerToRoman
- Definisce le rappresentazioni ASCII art per ogni carattere romano (I, V, X, L, C, D, M)
- Costruisce la rappresentazione ASCII art riga per riga
- Restituisce la rappresentazione ASCII art completa come stringa

//// FILE ////

Questa è la classe di test IntegerToRomanTest.java che verifica la funzionalità di conversione da numeri arabi a romani. La classe:

- È documentata per spiegare i principi A-TRIP e RIGHT-BICEP seguiti
- Testa input invalidi per verificare la gestione degli errori
- Utilizza test parametrizzati per verificare numerose conversioni fondamentali
- Testa conversioni più complesse con combinazioni di simboli
- Verifica i casi limite (1 e 1000)
- Utilizza le asserzioni di JUnit per verificare i risultati attesi

//// FILE ////

Questa è la classe di test `RomanPrinterTest.java` che verifica la funzionalità di stampa dei numeri romani in ASCII art. La classe:

- È documentata per spiegare i principi A-TRIP e RIGHT-BICEP seguiti
- Testa input invalidi per verificare la gestione degli errori
- Verifica la rappresentazione ASCII art corretta per simboli singoli (I, V, X, L, C, D, M)
- Verifica la rappresentazione ASCII art per combinazioni di simboli (IV, IX)
- Utilizza test parametrizzati per verificare che tutti i numeri base producano output non vuoto
- Verifica i casi limite (1 e 1000)
- Testa direttamente il metodo privato `printAsciiArt` usando reflection
- Utilizza le asserzioni di JUnit per verificare i risultati attesi

Spiegazione dell'Implementazione del Progetto

Ora che ho presentato tutti i file necessari per il progetto, ecco una spiegazione di come procedere per completare l'implementazione:

1. Configurazione del Repository:

- Crea un nuovo repository su GitHub
- Clona il repository localmente
- Configura l'ambiente Git secondo le istruzioni fornite

2. Gestione del Progetto:

- Crea milestone per la versione 1.0
- Configura la project board con colonne To Do, In Progress, Done
- Crea le issue per le diverse attività (come numerato nella guida)
- Assegna le issue ai membri del team

3. Sviluppo:

- Segui l'approccio TDD (Test-Driven Development):
 1. Scrivi prima i test
 2. Verifica che falliscano
 3. Implementa il codice minimo per farli passare
 4. Refactoring
- Implementa i requisiti in modo incrementale, partendo dai primi 3 numeri fino a 1000
- Segui il workflow Feature Branch (un branch per ogni feature)

4. Continuous Integration:

- Configura GitHub Actions per eseguire automaticamente build e test
- Configura Coveralls per tracciare la copertura del codice
- Assicurati che i badge nel README si aggiornino correttamente

5. Quality Assurance:

- Assicurati che tutti i test passino

- Verifica che la copertura del codice sia $\geq 85\%$
- Verifica che l'analisi statica (checkstyle) non riporti errori

Il progetto richiede una rigorosa attenzione alla qualità del codice e alle pratiche di sviluppo software, prestando particolare attenzione ai principi A-TRIP e RIGHT-BICEP per i test.